P2992R1
Attribute [[discard("reason")]]

Published Proposal, 2024-01-19

Author:
Giuseppe D'Angelo

Audience:
EWG, SG22

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose a new standard attribute, `[[discard("reason")]]`, to express the explicit intent of discarding the result of an expression.

Table of Contents

§ 1. Changelog

- R1
  - Split the proposal of the new attribute (this paper) from proposing attributes on expressions.
  - Change audience from EWGI to EWG.
- R0
  - First submission

§ 2. Motivation and Scope

[P0068R0] (and its subsequent [P0189R1] and [P1301R4]) introduced the `[[nodiscard]]` attribute in C++17; [WG14-N2267] introduced the same attribute in C23.

The main use case for the `[[nodiscard]]` attribute is to mark functions (and types returned by functions) whose return value is deemed important, and discarding it can have important consequences.

Examples of such functions include:

- **Pure functions**, that is, functions with no side effects, that are just called for their return value (example: `std::vector::empty()`). If the return value is not examined, it does not make sense to call the function at all.
  This is not merely a performance issue, but also a correctness one: functions and types that are poorly named (again, `empty()`) may mislead the user into thinking that the function actually does something (it *empties* the vector).
- **Functions that return error codes** or similar values that *must be examined* (and therefore used) for the calling code to be correct.
  In this case, discarding the return value constitutes a logical bug in user code, and encouraging a warning from the implementation should make the user aware of their mistake.
- **Functions that return unmanaged objects** that must be reclaimed in a special way (example: `std::allocator<T>::allocate`, that returns a pointer allocated with `operator new`).
  For these functions, accidentally discarding the return value incurs in resource leaks.
- **Functions that return manager objects** that should be kept alive; for instance, RAII managers that need to be kept alive in the block in which the function is invoked.
  Discarding the return value does not constitute any resource leak *per se*, but code "close" to the function call (e.g. in the same block) may operate under the false assumption that the manager (and thus the resource) is still alive/acquired.
  Examples of these functions are (after [P1771R1]) constructors of lock managers, where the constructed object itself should not get discarded, otherwise the following code would operate without the necessary mutex protection.

Still: in some scenarios it is useful to expressely discard the result of an expression, without having the implementation generating a warning in that case. Examples include:

- **Partial domains**. A nodiscard function that returns an error code may also document that it will never fail under certain conditions (for instance, when certain values are passed as parameters). If the user is already checking that the parameters are in the non-failing domain, there is no need to use the return value; it can be safely discarded.
  One could argue that the code could still consume the error code, and for instance `assert` on it; but, in general, we may simply not be interested in *enforcing a contract* on the function. This is akin to testing the function itself; in other words, to verify that the function matches its documented semantics. This is not the job of the user of a function, but of the implementor of the function.

- **Legacy**. A function may return a status code to indicate success or failure; then, later, its implementation evolves in a way such that it never fails (example: `pthread_once` on Linux). Still, the function may decide to keep returning the status code to preserve API and ABI compatibility. The return value can always be safely discarded as it's meaningless.
- **Testing**. We may want to call a nodiscard function in order to e.g. perform smoke testing, make sure that wide-contract functions don't crash, and similar. In these cases we are simply not interested in the return value, yet we do not want a warning.

This paper proposes a standard way to express the user's intent to discard the value of an expression. Such a way is currently missing, and workarounds are employed instead.

§ 2.1. What's a discarded return value?

A "discarded return value" is formally defined as a *discarded-value expression* in [expr.context]/2. They are:

- expressions appearing as *expression-statement* productions ([stmt.expr]/1);
- the left-side expression of the builtin comma operator ([expr.comma]/1);
- expressions explicitly converted to (cv-)void ([expr.static.cast]/6).

As per [dcl.attr.nodiscard]/4, implementations are encouraged to emit warnings when a *nodiscard call* (a call to a function marked `[[nodiscard]]` or returning an object of a type marked `[[nodiscard]]`) appears in the code and the result is not cast to `void` .

§ 2.2. Existing discarding strategies

There are currently two main strategies for discarding a value returned from a `[[nodiscard]]` function without producing a warning by the implementation.

§ 2.2.1. Cast to `void`

[dcl.attr.nodiscard]/4 states:

> Appearance of a nodiscard call as a potentially-evaluated discarded-value expression ([expr.prop]) is discouraged unless explicitly cast to `void`. Implementations should issue a warning in such cases.

This means that an explicit cast to `void` can be used to suppress the nodiscard warning:

```
[[nodiscard]] int f(int i);

f(123);         // warning here
(void) f(123);  // no warning
void(f(123));   // no warning
```

Using a cast to suppress the warning has several major shortcomings:

- It is a total abuse of notation. A cast notation is used, although one does not intend perform any type conversion; the code is just using an "arcane" language rule (any expression can be converted to *cv* `void`, [expr.static.cast]/6), as well as the explicit "concession" in the semantics of the `[[nodiscard]]` attribute, in order to suppress the warning.
- It is hard to explain to language novices why the cast works and what it means.
- It is hard to grep for in a codebase. Also, different codebases use slightly different spellings for the cast (`void(x)`, `(void)x`, etc.). As a workaround, some codebases hide the cast behind a macro, to better express the intent and make it greppable again.
- The very same notation is also used for `[[maybe_unused]]` semantics, that is, "this object may be unused in this scope". Again, some codebases hide this *different* intent with a different macro. Therefore, grepping may yield false positives.
- The rationale for the discard can only be provided via comments, not in code, and therefore it's hard/impossible to enforce rules that mandate such a rationale in a codebase.

§ 2.2.2. Assignment to `std::ignore`

It is possible to utilize `std::ignore` as a "sink" object:

```
[[nodiscard]] int f(int i);

std::ignore = f(123); // return value isn't discarded => no warning
```

Nitpicking, at the moment `std::ignore` is a facility related to tuples and `std::tie` (see [tuple.creation]); the code above is not *guaranteed* to work, although it does work on all major implementations. [P2968R2] aims at standardizing the current behavior, and therefore give precise semantics to the code above.

This solution is also "blessed" by the C++ Core Guidelines ([ES.48]), which favor it over the cast to `void`.

Still, we claim that this solution has a number of shortcomings:

- It is not usable by generic code, since it requires the right-hand side of the assignment to yield a value. If the right-hand side expression evaluates to `void`, the code becomes ill-formed.
- It is not compatible with C.
- `std::ignore` is a library solution. We believe a language solution to the problem would be better suited for what's exquisitely a language issue.
- It is relatively verbose compared to the cast to `void`.
- Again, the rationale as of why we want to discard the result value can only be provided in comments.

§ 2.3. A new attribute

We find both strategies suboptimal, and therefore in this paper we are proposing a novel one: the `[[discard]]` attribute, that complements `[[nodiscard]]`:

```
// returns an error code
[[nodiscard]] int f(int i);

f(123);                                                    // warning h
[[discard("f always succeeds with positive inputs")]] f(123);  // no warnir
```

Compared with the previous solutions:

- `[[discard]]` is symmetrical to `[[nodiscard]]` in usage and syntax: `[[nodiscard("reason")]]` is placed on the function/type declaration that we do not want callers to ignore; `[[discard("reason")]]` is placed on the expression whose result one expressly want to discard. This makes it straightforward to learn and understand.
- It supports a reason, expressed right in the attribute. While a C++ compiler would not use the reason itself (because it **won't** generate a warning), having a reason available is useful for users and other tooling. Specifically:
  - since the whole `[[nodiscard]]` mechanism is opt-in, if a function or a type is marked as such, it means that someone went the extra mile to warn us about a possible mistake we are

making (possibly with a message, as in `[[nodiscard("reason")]]`). Therefore, we would have a *structured* way to justify why we think that discarding the result is appropriate;
    - having a reason can also be enforced by tooling (for instance, code checkers could ban the usage of `[[discard]]` *without* a reason).
- It works with generic code (one can discard `void`).
- It does not require to pull components of the Standard Library for suppressing a language warning.
- It is perfectly compatible with C, and we would welcome its adoption by WG14 as well.
- It is only moderately more verbose than the cast to `void`.

2.4. Sampling existing codebases

A major compelling motivation for the `[[discard]]` attribute is to be able to express (and possibly enforce) *the reason* why the result of an expression is discarded.

Existing, widely used codebases feature cases where results are discarded without offering a motivation (in comments or commit messages). This makes it hard to understand "after the fact" if the code is correct, and what the intentions of the authors were.

Here's a few examples found in the wild where we think using `[[discard]]` would improve the existing code.

2.4.1. Google Test

In `gtest.cc` there is this snippet:

```
// In debug mode, the Windows CRT can crash with an assertion over invalid
// input (e.g. passing an invalid file descriptor).  The default handling
// for these assertions is to pop up a dialog and wait for user input.
// Instead ask the CRT to dump such assertions to stderr non-interactively.
if (!IsDebuggerPresent()) {
  (void)_CrtSetReportMode(_CRT_ASSERT,
                          _CRTDBG_MODE_FILE | _CRTDBG_MODE_DEBUG);
  (void)_CrtSetReportFile(_CRT_ASSERT, _CRTDBG_FILE_STDERR);
}
```

It is not clear at all why the return values are being explictly discarded; the functions themselves are not marked `[[nodiscard]]` (they are C functions, part of Windows' CRT), and removing the casts to void compiles without any warnings. Maybe some static analysis tooling raised a warning about the discarded return value? Maybe it's an internal code convention? We cannot know for sure; `[[discard]]` with an explanation would help.

2.4.2. libfmt

The only occurences of discarded values in libfmt are in testing code, where `fmt::format` itself (a `[[nodiscard]]` function) is being tested:

```
EXPECT_THROW_MSG(
    (void)fmt::format(runtime("{0:+}"), reinterpret_cast<void*>(0x42)),
    format_error, "invalid format specifier");
```

Here we want to discard the return value anyhow because we want to test that the call throws a given exception. The cast could be replaced by something like `[[discard("testing exceptions")]]`.

2.4.3. Qt

```
QDuplicateTracker<QString> known;
(void)known.hasSeen(path);
do {
    // ...
    // more code that uses `known`
    // ...
} while (separatorPos != -1);
```

This code is discarding the result of a `[[nodiscard]]` call, because it knows its outcome (since `known` has just been built, `hasSeen` will return `false`). What a casual reader may fail to realize is that *`known.hasSeen(path)` actually modifies `known`* by making it remember that it has seen `path` (in other words, the code is somehow initializing `known` before using it in the loop).

The alternative formulation is much more expressive:

```
QDuplicateTracker<QString> known;

[[discard("Loop invariant: the path is known by the tracker")]]
    known.hasSeen(path);
```

This is another example:

```
Qt::Orientation QGridLayoutEngine::constraintOrientation() const
{
    (void)ensureDynamicConstraint();
    return (Qt::Orientation)q_cachedConstraintOrientation;
}
```

`ensureDynamicConstraint` does some calculations, and returns a `bool` indicating whether it has succeded. The result of these calculations is stored (cached) in member variables. So why is this code discarding the success flag? The code doesn't say; maybe this function has preconditions, and one must first check that the calculations can succeed, and then one can call `constraintOrientation()`. But then why not asserting on the result of calling `ensureDynamicConstraint()`? We can only speculate; this code has been written over 15 years ago.

§ 2.4.4. Boost.Asio

The `posix_mutex` class has this code in its body:

```
class posix_mutex {
    // [...]

    // Destructor.
    ~posix_mutex()
    {
      ::pthread_mutex_destroy(&mutex_); // Ignore EBUSY.
    }

    // Lock the mutex.
    void lock()
    {
```

```
      (void)::pthread_mutex_lock(&mutex_); // Ignore EINVAL.
    }

    // Unlock the mutex.
    void unlock()
    {
      (void)::pthread_mutex_unlock(&mutex_); // Ignore EINVAL.
    }
};
```

What do the comments refer to? One may assume they refer to the reason behind discarding the return value of `pthread_mutex_lock` and `pthread_mutex_unlock`; for whatever reason, this class does not care for handling the error.

But then, why isn't the destructor also discarding the return value of `pthread_mutex_destroy`?

This raises the suspicion that the comments may not refer to the reason of the discarding, but to a some more general property of the operations: the error *cannot* be generated given the class' invariants. This would be something better expressed via assertions/contracts.

§ 3. Design Decisions

§ 3.1. Is `[[discard]]` an attribute on expressions or statements?

Based on the current usages of casting to `void` that we have sampled, the most natural use of `[[discard]]` is going to look like this:

```
// function call whose result we want to discard
[[discard("f never fails with positive numbers")]] f(42);
```

In this case the C++ grammar is already ruling that the attribute appertains to the **statement**. For this reason we are proposing that the `[[discard]]` attribute should first and foremost be applicable to an *expression-statement*.

If that expression contains multiple discarded-value expressions (by means of operator comma), the attribute will apply to them all, suppressing all their possible warnings:

```
[[discard]] a(), b(), c();   // don't generate discarding warnings
```

✧ ✧ ✧

What about applying `[[discard]]` to individual sub-expressions? Unfortunately at the moment the C++ grammar does not allow for attributes on expressions, and therefore we are unable to provide support for those cases.

We are authoring a different proposal (P3093) that would enable attributes on expressions; with that proposal, in principle one could write:

```
a(), ([[discard]] b()), c();
```

and only suppress discarding warnings for the call to `b`.

Supporting `[[discard]]` on expressions is somehow a secondary goal, because the only practical applications of such a discarding mechanism exists in the context of using the builtin comma operator (at least, to the best of our knowledge). We strongly believe that usages of the builtin comma operator should be frowned upon, except in corner cases where it's unpractical to use alternatives.

Still, some scenarios where `[[discard]]` on expressions would be nice to have are:

- in a constructor's member initialization list;
- as the increment (third) parameter of a `for` loop.

For instance:

```
for (int i = 0; i < N; ++i, ([[discard]] f(i)))
    doSomething(i);
```

We therefore plan to add support for discarding expressions, should attributes on expressions become possible.

§ 3.2. Can `void` be discarded?

Yes. We believe that such a situation can happen in practice, for instance in generic code, and such a restriction sounds therefore unnecessary and vexing.

§ 3.3. What should happen if `[[discard]]` is applied to an *expression-statement* whose expression isn't discarded-value?

Or, more in general, if it is applied to a statement that does not contain a nodiscard call?

For example:

```
[[nodiscard]] int f();

[[discard]] a = f(); // This is not *actually* discarding. Should it be leg


bool g();        // This is not marked as [[nodiscard]]
[[discard]] g(); // Legal?
```

Should we accept or forbid these usages, as the attribute is meaningless (at best) or misleading (at worst)? We are proposing to **accept** the code, under the rationale that the attribute serves to suppress a `[[nodiscard]]` warning. Since the warning would not be generated, there is nothing to suppress.

`g()`'s example is actually very compelling: there's a number of reasons why `g()` might not have been marked as `[[nodiscard]]` (legacy code, C APIs, 3rd parties that we can't modify ourselves, etc.); yet it's "normally" important to examine its return value. By allowing the attribute, one can still document why we are not examining it in that particular circumstance.

Of course implementations can still diagnose "questionable" usages as QoI.

§ 3.4. Bikeshedding: naming

The most obvious name for the attribute that we are proposing is `discard`, to mirror the existing `nodiscard` attribute.

There is a possible objection to this name: this attribute does not actually "force" the value of an expression to be discarded:

```
[[discard]] result = f(); // nothing is discarded here
```

One could therefore prefer a more nuanced form, such as `may_discard` (along the lines of `maybe_unused`):

```
[[may_discard]] f();          // actually discarding
[[may_discard]] result = f(); // not discarding
```

We would like to request a poll to EWG.

§ 4. Impact on the Standard

This proposal is a core language extension. It proposes a new standard attribute, spelled `[[discard]]` or `[[discard("with reason")]]`, to mark *expression-statement*s whose expressions' results we want to expressly discard.

No changes are required in the Standard Library.

§ 5. Technical Specifications

All the proposed changes are relative to [N4971].

§ 5.1. Proposed wording

In [cpp.cond] append a new row to Table 21 ([tab:cpp.cond.ha]):

Attribute	Value
`discard`	`YYYYMML`

with `YYYYMML` determined as usual.

✧ ✧ ✧

In [dcl.attr.nodiscard] insert a new paragraph after 3:

> 4. A potentially-evaluated discarded-value expression ([expr.prop]) which is a nodiscard call and which is neither
>
> - explicitly cast to `void` ([expr.static.cast]), or
> - an expression (or subexpression thereof) of an *expression-statement* marked with the `discard` attribute ([dcl.attr.discard])
>
> is a *discouraged nodiscard call*.

Renumber and modify the existing paragraph 4 as shown:

> ~~4.~~ 5. *Recommended practice*: ~~Appearance of a nodiscard call as a potentially-evaluated discarded-value expression ([expr.prop]) is discouraged unless explicitly cast to `void`~~ . Implementations should issue a warning ~~in such cases~~ for discouraged nodiscard calls . [...]

And renumber the rest of the paragraphs in [dcl.attr.nodiscard].

Modify the Example 1 as shown (including only the lines for the chosen approach):

```
struct [[nodiscard]] my_scopeguard { /* ... */ };
struct my_unique {
  my_unique() = default;                                  // does not acqui
  [[nodiscard]] my_unique(int fd) { /* ... */ }           // acquires resou
  ~my_unique() noexcept { /* ... */ }                     // releases resou
  /* ... */
};
struct [[nodiscard]] error_info { /* ... */ };
error_info enable_missile_safety_mode();
void launch_missiles();
void test_missiles() {
  my_scopeguard();              // warning encouraged
  (void)my_scopeguard(),        // warning not encouraged, cast to void
    launch_missiles();          // comma operator, statement continues
  my_unique(42);                // warning encouraged
  my_unique();                  // warning not encouraged

  enable_missile_safety_mode(); // warning encouraged
  launch_missiles();


  [[discard]] my_unique(123);          // warning not encouraged
  [[discard("testing illegal fds")]]
               my_unique(-1);          // warning not encouraged
  [[discard]] my_unique();             // warning not encouraged

  [[discard]] my_unique(),
    enable_missile_safety_mode();      // warning not encouraged

}
error_info &foo();
void f() { foo(); }             // warning not encouraged: not a nodiscar
                                // the (reference) return type nor the fu
```

✧ ✧ ✧

Add a new subclause at the end of [dcl.attr], with the following content:

??? Discard attribute [dcl.attr.discard]

1. The *attribute-token* `discard` may be applied to an *expression-statement*. An *attribute-argument-clause* may be present and, if present, shall have the form:

(*unevaluated-string*)

2. *Recommended practice*: Implementations should suppress the warning associated with a nodiscard call ([dcl.attr.nodiscard]) if such a call is an expression (or subexpression thereof) of an *expression-statement* marked as `discard`. The value of a *has-attribute-expression* for the `discard` attribute should be `0` unless the implementation can suppress such warnings.

   The *unevaluated-string* in a discard *attribute-argument-clause* is ignored.

   [*Note* 1: the string is meant to be used in code reviews, by static analyzers and in similar scenarios. — *end note*]

6. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

References

Informative References

[ES.48]
Bjarne Stroustrup; Herb Sutter. *C++ Core Guidelines, ES.48: Avoid casts*. URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-casts

[N4971]
Thomas Köppe. *Working Draft, Programming Languages — C++*. 18 December 2023. URL: https://wg21.link/n4971

[P0068R0]
Andrew Tomazos. *Proposal of [[unused]], [[nodiscard]] and [[fallthrough]] attributes*. 3 September 2015. URL: https://wg21.link/p0068r0

[P0189R1]

Andrew Tomazos. *Wording for [[nodiscard]] attribute*. 29 February 2016. URL: https://wg21.link/p0189r1

[P1301R4]

JeanHeyd Meneide, Isabella Muerte. *[[nodiscard("should have a reason")]]*. 5 August 2019. URL: https://wg21.link/p1301r4

[P1771R1]

Peter Sommerlad. *[[nodiscard]] for constructors*. 19 July 2019. URL: https://wg21.link/p1771r1

[P2968R2]

Peter Sommerlad. *Make std::ignore a first-class object*. 13 December 2023. URL: https://wg21.link/p2968r2

[WG14-N2267]

Aaron Ballman. *The nodiscard attribute*. URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2267.pdf